SAMPLE ASSIST · ENGINEERING · INTERN PROJECT

SA Marketplace — Front-end Hand-over Report

The customer marketplace, the seller portal and SA Admin: what was built, why, how to run it, how it was checked, and what comes next. It covers the three React apps only; the backend has its own delivery report.

Prepared for: Tech Lead · **Prepared by:** Rusira Senath Liyanage · **Status as at:** 27 October 2026 · **Source of truth:** Intern Project Brief, 15 July 2026

1. Summary

The brief asks for **one shared backend with several front-end entry points**. Three of the four entry points now exist as React apps on top of the Phase 1 backend: the **customer marketplace** for buyers, the **seller portal** for verified suppliers, and **SA Admin** for Sample Assist staff. Every screen in their designs is built and connected to the real API, with no sample data or fake backend, and each app was checked in a browser against the seeded database. The **Collection Manager (CM) store**, the fourth surface, has not been started. Nothing is deployed; everything runs locally, as the brief requires.

3	42	746	0
React apps (ports 5173, 5174, 5175)	designed screens built (13 + 18 + 11)	automated tests in 130 files (166 + 337 + 243)	accessibility violations found by axe on any checked page

Four things to know up front

1. **The CM store is not built.** It is blocked on two open decisions (how CM users sign in, and how CM inventory decreases), both in the backend report. The three built apps are the template for it (sections 8 and 9).
2. **Pages and the Redux store have no automated tests.** The 746 tests cover helpers, forms, tables, dialogs and layouts. Pages that call the API are checked in a browser against the real API, with a written walkthrough for the seller portal and SA Admin. An end-to-end suite is the main quality gap (section 6).
3. **About 35 generic files exist in three copies** (table, pagination, dialog, layouts, hooks, the API client) because a shared package was deliberately not built. A fix to one must go into all three. Whether to extract a package before a fourth app is an open decision (section 5).
4. **Where a design and the backend disagree, the backend wins.** Each app's plan lists every difference: 34 for the seller portal and 20 for SA Admin, with the customer app's in its plan and in section 3 below.

What you are receiving

> **Latest source code:** the `phase-1-completed` branch of the Bitbucket repository `innomente/equipment-assist`. The front ends live in `SA_MarketPlace/Frontend`: `customer-facing-frontend`, `seller-portal` and `admin-portal`. Run that branch; the pull-request split in `Backend/docs/report/pull-request-guide.md` (sections 6 and 7) sets out how to review and merge the front-end commits in order.

Item	Where to find it
Source code	`SA_MarketPlace/Frontend/<app>/src`, about 18,500 lines excluding tests
Setup instructions	Section 2 of this report; `Frontend/README.md` (all three apps) and each app's own `README.md`
Plans and screen lists	`docs/ui-build-plan.md` in each app: every screen, route and endpoint, and where the backend won over the designs
Browser walkthroughs	`docs/browser-checklist.md` in the seller portal and SA Admin: the steps, the accessibility pass and known limitations
State and API guide	`customer-facing-frontend/docs/state-and-api.md`: how the store and client work, and the recipe for a new feature
Tests	`__tests__` folders beside the code (`pnpm test` in each app)
This report	`Frontend/docs/report` (PDF, HTML source and the script that renders it)

Not included: the screen designs, which are kept in the project's separate design folders rather than the repository and should be shared with the team alongside the code; and any secrets. The front ends hold none: their only configuration is an API address. Every login is a fictional seeded account.

2. Set it up on your own machine

Allow about 15 minutes, most of it the backend's first install. The front ends need the backend running, because none of them has sample data.

2.1 What you need

Tool	Version	Notes
Node.js	24 or newer	Each app has an `.nvmrc` (`nvm use`); `engines` enforces it. The tests' jsdom does not run on Node 22 or older. Checked on 25.0.0
pnpm	10.17.1	Pinned in each `package.json`; `corepack enable` installs it
Docker Desktop	any recent	For the backend's Postgres only. Must be running
Free ports	5173, 5174, 5175	The front ends, plus the backend's 5433, 8001, 8002 and 8003. The front-end ports are fixed because the backend's reset-password links point at them

2.2 Start the backend, then a front end

1. **Start the backend (database, migrations, seed data, three APIs)**

```
cd SA_MarketPlace/Backend
cp .env.example .env
pnpm install
pnpm db:up && pnpm --filter @sa-marketplace/entities migration:run && pnpm db:seed
pnpm dev
```

`pnpm dev` runs all three APIs in one terminal. Use it or the three single commands (`pnpm customer-facing`, `supplier-api`, `admin-api`), never both: they share ports and build folders and restart each other in a loop.

2 **Start a front end, one terminal each for the ones you want to see**

```
cd SA_MarketPlace/Frontend/customer-facing-frontend   # or seller-portal, or admin-portal
pnpm install
pnpm dev
```

No `.env` is needed: the default API address is the local backend. Copy `.env.example` to `.env` only to change `VITE_API_URL` (the seller portal also has `VITE_SUPPORT_EMAIL`).

3 **Open VS Code on the app's own folder**

Not the repository root: the root window pins the Backend's TypeScript 4.9, which cannot read the front ends' config and shows false "cannot find module" errors.

2.3 The three apps and their logins

Every seeded account uses the password `Password123!`. Running `pnpm db:seed` in `Backend/` puts the data back after testing.

App	Address	API	Sign in with
Customer marketplace	localhost:5173	8001	`j.shaw@saclinics.com.au` (buyer and collection manager); `s.chen@miningco.com.au` and `t.beckett@worksafe.com.au` have order history. Test cards: `4242 4242 4242 4242` approves, `4000 0000 0000 0002` declines
Seller portal	localhost:5174	8002	`priya@bindimedsupply.com.au` or `m.webb@safetyfirstdist.com.au` (approved); `r.patel@clinicalsupplies.com.au` (pending); `d.liu@cheaptestkits.com.au` (rejected)
SA Admin	localhost:5175	8003	`a.morgan@sampleassist.com.au`. A new admin account needs a `@sampleassist.com.au` address

2.4 Commands, in each app's folder

Command	What it does
`pnpm dev`	Dev server with hot reload
`pnpm test`	Vitest and Testing Library (`pnpm test:watch` to watch)
`pnpm typecheck`	`tsc -b`
`pnpm lint`	ESLint
`pnpm build`	Type check, then a production build into `dist/` (`pnpm preview` serves it)

2.5 A ten-minute walkthrough that crosses all three apps

1. **Buyer.** In the customer app sign in as `j.shaw@saclinics.com.au`. Search the shop, filter by category and price, open a product, add two items from different sellers (one from Bindi MedSupply), and check out with the declined card (the cart stays intact), then with the approved card. A cart with two sellers becomes two orders.
2. **Seller.** In the seller portal sign in as `priya@bindimedsupply.com.au`. The new order is on the dashboard with an SLA countdown. Open it, tick the pick list, and dispatch; the invoice can then be downloaded.

3. **Buyer again.** In the customer app, My orders now shows the order as dispatched, with the same invoice PDF.
4. **Admin.** In SA Admin sign in as `a.morgan@sampleassist.com.au`. Orders lists every order across sellers, with CSV export and the same invoice.
5. **Verification.** In the seller portal register a new business. It lands on "under review". In SA Admin open Suppliers, review it, tick the three checks (Approve unlocks), and approve. Sign in to the seller portal again: the seller sees the approval notice once, then the dashboard. Reject a different one to see the reason on the seller's screen.

> **Approve and reject send an email to the applicant and cannot be undone** from the screens (there is no suspend in Phase 1). The pop-ups say so. With no email settings the message is printed in the API terminal instead.

3. What the brief asked and what was delivered

The brief's Phase 1 lists the capabilities below. The rows show which front end delivers each one; the backend report covers the API side.

Brief item	Status	Front-end delivery
Accounts and roles	Done	Sign-in, registration, forgot and reset password, session handling and route guards in all three apps. Each app talks only to its own API and its token is rejected by the others. SA Admin sign-up is limited to staff email domains.
Browse and search	Done	Customer app: keyword search, category and price-range chips that combine, sorting, paging with a 10, 20 or 30 page size, and a detail page with suggestions.
Cart and checkout	Done	Live-priced cart with steppers that appear only once an item is in the cart; two-step checkout; mock card payment with a declined-card pop-up.
Orders	Done	Buyers: history, status filters, search, detail and reorder. Sellers: dashboard queue, orders table, fulfilment with pick list, dispatch and cancel with a reason. Admins: all orders, filters, sorting, CSV export.
Electronic invoice	Done	"Download invoice" in all three apps serves the same stored PDF, offered once an order has dispatched. The file keeps its name.
Seller catalogue	Done	List with search, status chips, sorting and a low-stock banner; add and edit with up to four images; Save draft and Publish; Retire (final).
Seller onboarding and verification	Done	Seller sign-up with a checksum-validated ABN and the pending, rejected (with resubmit) and approved screens; admin queue, application review, approve and reject with a reason the seller sees.
CM ordering	Not started	The CM store front end, and a decision on single sign-on.
Inventory update	Backend only	Placing an order reduces the supplier's stock. There is no screen for the buyer's own inventory; it needs the deferred "received" step (backend report).

The customer marketplace

Thirteen designed screens, all built: catalogue, product detail, cart, checkout (delivery and payment), order confirmation, My orders, order detail, add-to-cart states, items-per-page dropdown, login, register and edit profile.

Topic	What was done
Stack and structure	React 19, Vite, TypeScript, Ant Design 6, styled-components reading the Ant Design tokens through `ThemeBridge`, Redux Toolkit with RTK Query in the CM's feature-folder shape (`actions`, `slice`, `selector`, `types`). A store used by one page lives in that page's folder.
API and sessions	One client turns every failure into one error shape and shows the backend's own message. The token lives in `sessionStorage`; a rejected token ends the session with "Session expired"; sign-out clears all cached data. Guards send visitors to sign in and back.
Where the backend won	GST is the total divided by 11; no supplier rating; the backend's password rule, card rules and statuses; no current-password field on change password; no PO number; invoice offered only after dispatch; no "Notify me" (it promised an alert that never came).
Backend changes it needed	An order-number `search` on `GET /orders`, the invoice file name exposed over CORS, and optional real email. All small, additive and tested.
Colour and accessibility	The brand blue was darkened to `#0958d9` so white text passes WCAG AA (6.5 to 1). axe runs in a test on every page and pop-up; skip link, one `h1` per page, announced page titles, named dialogs, labelled fields.

The seller portal

Eighteen designed screens, all built, plus a reset-password page and a retire pop-up that the designs did not include. A single gate sends a seller to the right place after sign-in: the portal, or the screen that says where their application stands.

Topic	What was done
Stack and structure	Built from the customer app with a left sidebar (a menu button and drawer below 992 px) and the seller's orange. The dashboard shows live counts and the orders to dispatch, closest deadline first.
Where the backend won	34 differences are listed in the plan. For example: no "product categories" or "role" box on sign-up (the profile derives categories from products), dispatch sends no email, status screens say "Sample Assist", and sellers set prices directly.
Colour and accessibility	White on the brand orange is only 2.4 to 1, so buttons use near-black text and orange text is the darker `#ad4e00`. axe found four issues in the final pass (unlabelled read-only fields, a details list without terms, colour-only links, placeholder contrast); all fixed.
Behaviour worth knowing	Confirm dispatch stays disabled until every pick-list item is ticked, with a warning that says why. Price and stock are read-only in the catalogue list and changed on the edit page. A failed dispatch from the queue is shown, never silent.
Backend change it needed	One line: `Content-Disposition` exposed over CORS so the invoice keeps its name.

SA Admin

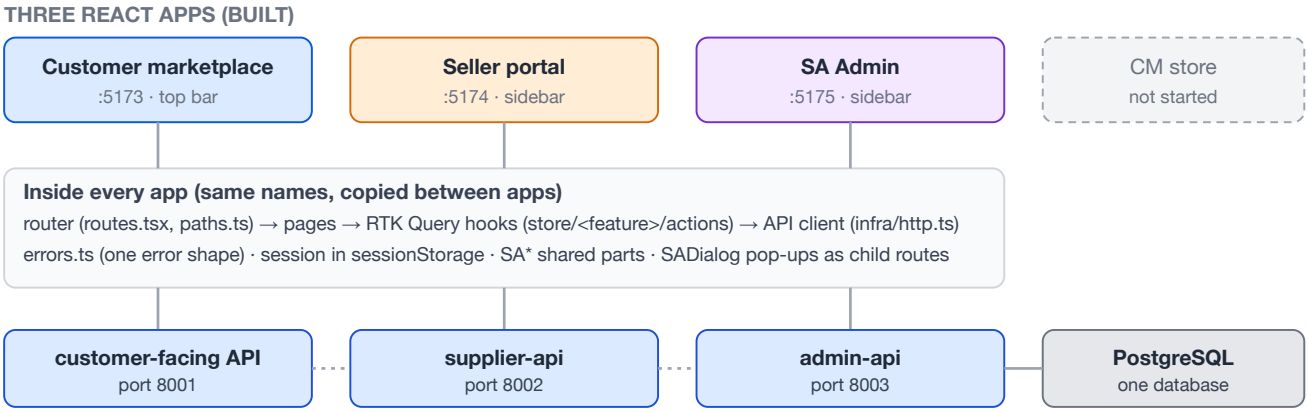
Eleven designed screens, all built: sign-in, register, supplier verification (queue, empty queue), application review with approve and reject pop-ups, all orders (and its empty state), order detail and My profile, plus the forgot and reset password pages.

Topic	What was done
Stack and structure	Built from the seller portal with its own purple and a sidebar of Suppliers (with a pending-count badge), Orders and My Profile.

Topic	What was done
Where the backend won	20 differences are listed in the plan. The main ones: no "keep me signed in"; no invitation code on register (the backend requires a terms tick and a staff email domain); only the name is editable on the profile; no "Invite administrator" card; order filters use the backend's real payment statuses and fulfilment stages.
The verification checklist	Three tick boxes on a pending application. **Approve is disabled until all three are ticked**, with a notice beside the boxes saying why; Reject is always available. The ticks are not saved, because the backend has no field for them, so this only guards against an accidental approval and does not prove the checks were made.
Decisions it makes safe	Both decisions email the applicant and cannot be undone here, and the pop-ups say so. A decided application shows who decided it, when, and the reason given. If another admin decides first, the screen shows the backend's 409 message and refreshes.
Backend change it needed	One line: `Content-Disposition` exposed over CORS so the CSV export and the invoice keep their names.
Accessibility	The purple `#722ed1` is 6.9 to 1 against white and needed no workaround. axe found no violations on 20 signed-in pages and pop-ups, or on the five public pages.

4. Architecture

Each app is an independent single-page application with the same shape. A page reads the store through RTK Query hooks; the hooks call one API client; the client calls that app's API and nothing else.



The CM store, when built, uses the customer-facing API: same account, orders and inventory.

Folder shape, the same in all three

Folder (under `src`)	What lives there
`router/`	`routes.tsx` declares the whole route tree once (pages load lazily); `paths.ts` holds every URL, so a route change is one file; `handle.ts` gives each route its title and breadcrumbs
`pages/`	One folder per screen, each with its own `components/` and `__tests__/`. `Modals/` holds the pop-ups, each with its own route
`components/`	Shared `SA`-prefixed parts (`SATable`, `SAPagination`, `SADialog`, `SAPageHeader`…) and `layouts/`. Anything used on one page stays in that page's folder; two or more moves here

Folder (under `src`)	What lives there
`store/`	Redux features (`actions` = RTK Query endpoints, `slice` , `selector` , `types`) and the store setup
`infra/`	The API client (`http.ts` , `api.ts` , `errors.ts`) and `pagination.ts` , the backend's paging contract
`hooks/ utils/ patterns/ constants/ styles/`	Helpers; `utils/format.ts` formats money, dates and counts; `styles/theme.ts` is the app's Ant Design token set

Routes at a glance

App	Routes
Customer	`/shop` , `/shop/products/:id` , `/cart` (+ remove and clear pop-ups), `/checkout/delivery` , `/checkout/payment` (+ declined), `/checkout/confirmation/:order` , `/orders` , `/orders/:order` (+ reorder), `/profile` (+ sign out); public: `/login` , `/register` , `/forgot-password` , `/reset-password` , `/session-expired`
Seller	`/` (dashboard), `/application` and `/application/resubmit` (before approval), `/orders` , `/orders/:order` (+ cancel), `/catalogue` , `/catalogue/new` , `/catalogue/:id` , `/catalogue/retire/:id` , `/profile` (+ sign out); the same public pages
Admin	`/suppliers` , `/suppliers/:id` (+ approve, reject), `/orders` , `/orders/:order` , `/profile` (+ sign out); the same public pages
All	`/403` , `/500` and a not-found page

5. Decisions and why

These shape the front ends most. The seller portal and SA Admin keep the full list in their `ui-build-plan.md` (section 8, "Decisions (settled)").

Decision	Why	Industry practice followed
Ant Design is the only UI library; one theme token set per app	The brief says marketplace, seller and admin use Ant Design with a distinct primary colour each, so the surfaces read as separate products. A colour change is one token.	Design tokens; a single component library.
styled-components reads the Ant Design tokens	Styles sit at the bottom of the component file as in Collection Manager, and both libraries read one theme, so nothing drifts.	Design-token theming; co-located styles.
RTK Query for every API call	Caching, loading and error states and re-fetching on change come from one tool, in the feature-folder shape the CM already uses.	Server state kept apart from UI state.
No sample data, no fake backend	A mock hid real mismatches. Removing it meant every screen was checked against the real API and its real errors.	Test against the real contract.

Decision	Why	Industry practice followed
The backend wins over the designs	The API enforces the rules (GST, passwords, statuses, who may do what). A screen that promised otherwise would mislead, so each difference is written down per app.	One source of truth for business rules.
Pop-ups are routes	A pop-up is a child route of the page it opens over (`/cart/remove/:id`), so Back closes it and a link can open it. All use `SADialog` : an icon, a one-line summary, what changes, what stays the same, and equal buttons with the safe choice first.	URL as state; consistent confirm-dialog pattern.
Ask before losing edits	Pages that edit saved data (customer profile, seller product form and business profile, admin profile) use `useUnsavedChanges` : leaving asks to keep editing, discard or save; closing the tab gets the browser's own warning.	Unsaved-changes guard on forms.
Never disable a control without saying why nearby	A greyed button with no reason is a dead end. Confirm dispatch and Approve supplier each carry a notice that says what unlocks them.	WCAG: operable, understandable controls.
The access token lives in `sessionStorage` ; sessions are 15 minutes with no renewal	It ends with the tab, and the backend revokes tokens on sign-out and password change. "Keep me signed in" was left out because the backend does not support it (ADR 0045), and showing it would mislead.	Short-lived tokens; do not offer what the API cannot do.
Copy the generic files, do not share a package	The brief says the CM and marketplace deliberately share no component library, and a package for one team's first three apps was more machinery than the work needed. Cost: three copies of about 35 files (a fix goes into all three). **Open decision** before a fourth app.	Rule of three for extraction.
Latest releases; TypeScript 7 compiles	The apps start on current versions (`pnpm outdated` prints nothing). `typescript` is aliased to the 6.0 package so `typescript-eslint` can read it until it supports 7.	Stay current; keep the toolchain working side by side.
No enums	`erasableSyntaxOnly` is on, so `as const` objects and string unions replace enums.	Types that erase to plain JavaScript.

Questions the brief left open, and how they were settled on the front end

Question	Answer
Seller and buyer status words differ	One backend status, shown in each app's own words (backend ADR 0036).
When a seller sees the "approved" screen	Once, on the first sign-in after approval; the seller portal's gate handles it (backend ADR 0032).
Changing the sign-in email	Not offered: the seller profile says so beside the field. Re-verification of the new address would be needed first.
Where seller prices are set	Directly by the seller on the edit page, with no approval loop (brief). Read-only in the list so a price is never changed by accident.

Question	Answer
CM sign-in and CM inventory	**Still open** (section 7).

6. Quality and accessibility

Automated checks (run for this report)

Type check, lint, unit tests and a production build were run on each app on 27 October 2026. All pass with no errors.

App	Test files	Tests	Type check	Lint	Build
Customer marketplace	35	166	Pass	Pass	Pass
Seller portal	51	337	Pass	Pass	Pass
SA Admin	44	243	Pass	Pass	Pass

What the tests cover: pure helpers (query builders, status mapping, formatters, change detection, validation rules), forms, tables, the pop-ups and layouts, and, in the customer app, an axe run over every page and pop-up. **What they do not cover:** pages that call the API and the Redux store, because the fake backend was removed. Those are checked in a browser against the real API (below). This is the largest gap in the front end (section 7).

Browser verification

App	How it was checked
Customer marketplace	Walkthrough against a freshly seeded database: sign-in, shopping, cart, checkout declined and approved, orders, invoice, reorder, profile (including a password change and restore), sign-out and registration.
Seller portal	The 16-step walkthrough in `docs/browser-checklist.md`, including the dispatch gate, the retire and cancel pop-ups, the product form and the business profile.
SA Admin	The 19-step walkthrough in `docs/browser-checklist.md`, with decisions made in the real database (a supplier rejected, another approved and checked from the seller side). It found two bugs the tests had missed: a previous record showing after a bad link, and a crash when the profile arrived after the page first rendered. Both are fixed with regression tests confirmed to fail without the fix, and the second was also fixed in the customer app.

Accessibility

axe-core (WCAG 2.1 A and AA, colour contrast included) found no violations on any page or pop-up checked, after the fixes noted in section 3. Every page has one `h1`, an announced title, a "Skip to main content" first tab stop and named dialogs. Below 992 px the navigation becomes a menu button and drawer that a keyboard can reach. **Not covered:** a screen-reader pass with a real reader, and responsive page layouts.

App	Primary colour	Contrast with white	Treatment
Customer	`#0958d9`	6.5 to 1	Darkened from the designs' `#1677ff` (4.1 to 1)
Seller	`#fa8c16`	2.4 to 1	Near-black text on orange fills; orange text is `#ad4e00`
Admin	`#722ed1`	6.9 to 1	None needed

Security on the front end

Concern	Control
Who can see what	The API decides; the front end filters nothing itself. A route guard sends visitors to sign in, and the seller portal's gate keeps unapproved sellers off the portal. Another user's record answers 404, which the pages show as "not found".
Tokens and secrets	No secrets in any front end. The token is in `sessionStorage`, sent as a Bearer header, and cleared on 401 or sign-out. `.env` is git-ignored; `.env.example` holds only an API address.
Card data	The payment form posts to the mock API; nothing is stored in the browser, and the backend keeps only the last four digits and the brand.
Input	Forms validate (email, password rule, ABN checksum, lengths) for speed, and the backend validates again for safety, and its messages are shown as written. React escapes everything it renders; no HTML is injected.

7. Known limitations and open items

Item	State and what is needed
CM store front end	Not started. Needs the two decisions in the backend report (single sign-on, and how CM stock decreases).
No page or store tests	Covered by browser walkthroughs only. A small end-to-end suite (for example Playwright) against the seeded API would close this for all three apps.
Pages are desktop-only	Below 992 px the navigation adapts, but tables scroll sideways or squeeze and cards crowd. Responsive layouts are Phase 2 (brief).
15-minute sessions	No refresh tokens (backend ADR 0045). A seller filling the long product form, or an admin editing a name, can be signed out and lose it. Saving a draft often is the workaround.
Admin sign-up is a speed bump	It only checks the email domain, so anyone can create `anything@sampleassist.com.au` and approve suppliers (backend ADR 0038). Fine for local fake data; replace it before any real use, for example an existing admin approving each new admin.
The admin checklist is not saved	It gates Approve on the screen but nothing records that the checks were made. Saving it needs a backend field.
Approve and reject cannot be undone	There is no suspend or offboarding in Phase 1. The pop-ups say so before confirming.
The pick list is not saved	The ticks live on the screen; the backend has no "picking" status. A refresh forgets them.
Seeded product images do not exist	Cards show a category icon and the browser logs a failed request per image. Products created in the seller portal show real pictures.
Backend gaps the screens expose	No "delivered" action; no storage conditions or shelf life on a seller's product form; no choosing a primary image; no "Notify me" when out of stock; no change of sign-in email; reset emails only if Gmail is configured.
Large exports	The backend refuses a CSV over 10,000 rows before sending anything; the screen shows its message. Narrow the filters.

Item	State and what is needed
Three copies of about 35 files	A fix goes into every copy. One bug fix already had to be made in two apps.

8. Next steps

#	Step	State and what it involves
1	**Foundation**	Done Vite and React project, typed API client with token handling, routing, layouts, shared components.
2	**Customer marketplace**	Done
3	**Seller portal**	Done
4	**SA Admin**	Done
5	**CM store**	Not started Catalogue, cart, checkout and My orders inside Collection Manager, matching its existing look (navy `#00004C`, blue `#156CC9`, not Ant Design), plus the sign-in decision. Each buyer feature is one backend ticket and two front-end tickets, because the two buyer surfaces share no component library.

Phase 2 and 3 on the front end (from the brief)

Phase	Candidates
Phase 2	Responsive and mobile layouts; richer filters and sorting; saved addresses; seller analytics and low-stock alerts; ratings and reviews; seller storefront pages; order tracking with notifications; customer-specific pricing screens in both buyer apps and a management screen for sellers; ABN lookup in the admin review.
Phase 3	Admin reporting dashboard; notifications centre; recurring orders; multi-user seller accounts; audit views; performance work.
Always out of scope	Real card processing, shipping-carrier integrations, native mobile apps, multi-currency, and production deployment.

9. Recommendations

In the order I would tackle them.

#	Recommendation	Why, and who
1	**Merge `phase-1-completed` into `main` through reviewed pull requests**	The brief wants work to land by pull request and never broken. Review in the order in `Backend/docs/report/pull-request-guide.md` (front-end commits in sections 6 and 7), not as one diff, and run the four checks in section 2.4 on each app at each step. *Tech lead.*
2	**Settle the two open decisions before building the CM store**	How CM users sign in, and how CM stock decreases. They shape the CM screens and the data flow. *Tech lead with the CM team.*

#	Recommendation	Why, and who
3	**Decide on a shared package before the fourth app**	Three copies of about 35 files is where copying stops paying off. Either extract them into a workspace package (`pnpm` workspaces, as the backend does) or keep copying with identical names and tests. The CM store will not use Ant Design, so only the non-visual parts (API client, errors, pagination, formatters, hooks) are candidates for sharing with it. *Tech lead.*
4	**Add an end-to-end suite**	It would replace the written walkthroughs and cover the pages and store that have no tests. Run it against a freshly seeded database.
5	**Replace admin sign-up before any real use**	Require an existing admin to approve each new admin, or verify the mailbox. *Tech lead.*
6	**Add responsive layouts when Phase 2 starts**	The navigation already adapts; the pages need work, buyer apps first.
7	**Revisit sessions with real use**	If the 15-minute sign-out hurts (long forms), add refresh tokens (design in backend ADR 0045) and an auto-saved draft in the product form.

Main risks

Risk	Level	Mitigation
CM sign-in and inventory rules are undecided	High	Recommendation 2 before any CM screens.
Pages and store untested by code	Medium	Browser walkthroughs now; an end-to-end suite (recommendation 4).
Three copies of shared files	Medium	Same names and tests everywhere; decide on a package (recommendation 3).
Admin sign-up only checks the email domain	Medium	Fine for local fake data; replace before real use (recommendation 5).
Desktop-only pages	Low	Out of Phase 1 by the brief; recommendation 6.
Knowledge held by one developer	Low	Reduced by the plans and walkthroughs in each app's `docs` , READMEs that run from a clean checkout, and this report.

Appendix A: package versions

The same in all three apps (`package.json`); everything is the latest release.

Package	Version	Role
react, react-dom	19.3	UI
vite, @vitejs/plugin-react	8.3, 6.1	Dev server and build
typescript	7.0 compiles	`typescript` is aliased to 6.0 so `typescript-eslint` works
antd, @ant-design/icons	6.6, 6.3	UI library and icons
styled-components	6.6	Styles reading the Ant Design tokens
@reduxjs/toolkit, react-redux	2.13, 9.3	State and RTK Query
react-router	8.5	Data router, `useBlocker` , child-route pop-ups
vitest, @testing-library/*, jsdom	5.0, 16 / 14 / 7, 30	Tests (Node 24 or newer)
eslint, typescript-eslint, react-hooks and react-refresh plugins	10.13, 8.72, 7.1, 0.5	Lint (the react-refresh rule means component files export only components)
@fontsource-variable/inter	5.3	The font, served locally

Appendix B: documents in each app

File	What it holds
`Frontend/README.md`	Setup for all three apps, logins, commands, shared rules, troubleshooting
`<app>/README.md`	Status, structure, conventions, routes (customer), versions
`customer-facing-frontend/docs/ui-build-plan.md`	The ten build steps, design points settled, accessibility pass
`customer-facing-frontend/docs/state-and-api.md`	Store layout, decisions, each connected feature, and the recipe for adding one
`seller-portal/docs/ui-build-plan.md`	Scope, screens, routes and endpoints, 34-row ledger of backend-over-design decisions, architecture, build steps, risks
`admin-portal/docs/ui-build-plan.md`	The same for SA Admin, with a 20-row ledger
`seller-portal/docs/browser-checklist.md` , `admin-portal/docs/browser-checklist.md`	The walkthrough steps, the axe pass record and known limitations